App Dev Project Report

1. Student Details

Name: Aviral Srivastava

Roll Number / Enrollment: 24F2005828

Email: 24f2005828@ds.study.iitm.ac.in

About Me

I am currently a third year computer science undergraduate student with a keen interest in web application development, backend architecture, and distributed systems. I focus on learning diverse modern technologies by designing real-world projects and understanding how enterprise software applications are structured, optimized, and deployed. My learning methodology prioritizes practical implementation, robust state management, and clean code principles rather than purely theoretical concepts. I continuously refine my engineering skills by building full-stack web platforms, debugging complex workflows, and integrating asynchronous background worker services.

---

2. Project Details

Project Title: Adv Placed (Placement Portal)

Problem Statement

Educational institutions face significant challenges in managing recruitment drives, student applications, and corporate partnerships efficiently. Traditional manual processes or fragmented communication channels often lead to data inconsistency, delayed interview scheduling, lack of application transparency, and tedious manual report generation.

The objective of Placed is to design and develop a comprehensive, multi-role web-based placement management portal that bridges the gap between Institution Administrators, Partner Companies, and Students. The system provides structured workflows for:

- **Admin Control:** Approving company registrations, managing student registries, monitoring placement metrics, and generating site-wide reports.
- **Company Drive Management:** Posting placement drives with custom eligibility criteria, tracking applicant submissions, updating interview dates/times, and shortlisting or selecting candidates.

- **Student Career Hub:** Browsing approved placement drives, maintaining personal profiles, uploading resumes, tracking application statuses, and receiving interview schedules.

Approach & Architecture

The project follows a decoupled, modular full-stack architecture:

- **Backend Service (Python & Flask):** Developed using Flask as a lightweight, high-performance RESTful API service. Functionalities are modularized into data models (models/), DTO serializers (dto/), route handlers (api.py), background worker tasks (tasks.py), and Redis session handlers (redis_client.py).
- **Frontend Application (Vue 3 & Vite):** Built using Vue 3 Reactive Components and styled with Vanilla CSS for a modern, fluid user interface. Component-driven views (AdminDashboard, CompanyDashboard, StudentDashboard, StudentDrives, StudentApplied) consume backend REST endpoints asynchronously.
- **In-Memory Caching & Background Jobs (Redis & Celery):** Integrated Redis (127.0.0.1:6379) for sub-millisecond session authentication caching and Celery for asynchronous background job processing (CSV data exports and HTML monthly report compilation).

---

3. AI/LLM Declaration

In this project, AI/LLM assistance tools (such as ChatGPT and AI pair programming assistants) were utilized during the software development lifecycle. The scope of AI usage included:

- **Coding & Boilerplate:** Assisting with routine code snippets, boilerplate REST endpoints, and CSS layout refactoring.
- **Debugging:** Debugging specific syntax errors, database relationship edge cases, and environment configuration issues.
- **Documentation:** Formatting technical documentation and structuring architectural reports.

Estimated AI/LLM Assistance: 25–35 percent of the overall work.

Note: *All core architectural design decisions, state management logic, business rules enforcement (e.g., active status restrictions, role-based authentication, database ORM relations), and end-to-end integration testing were performed independently by me. AI tools served purely as supportive coding accelerators.*

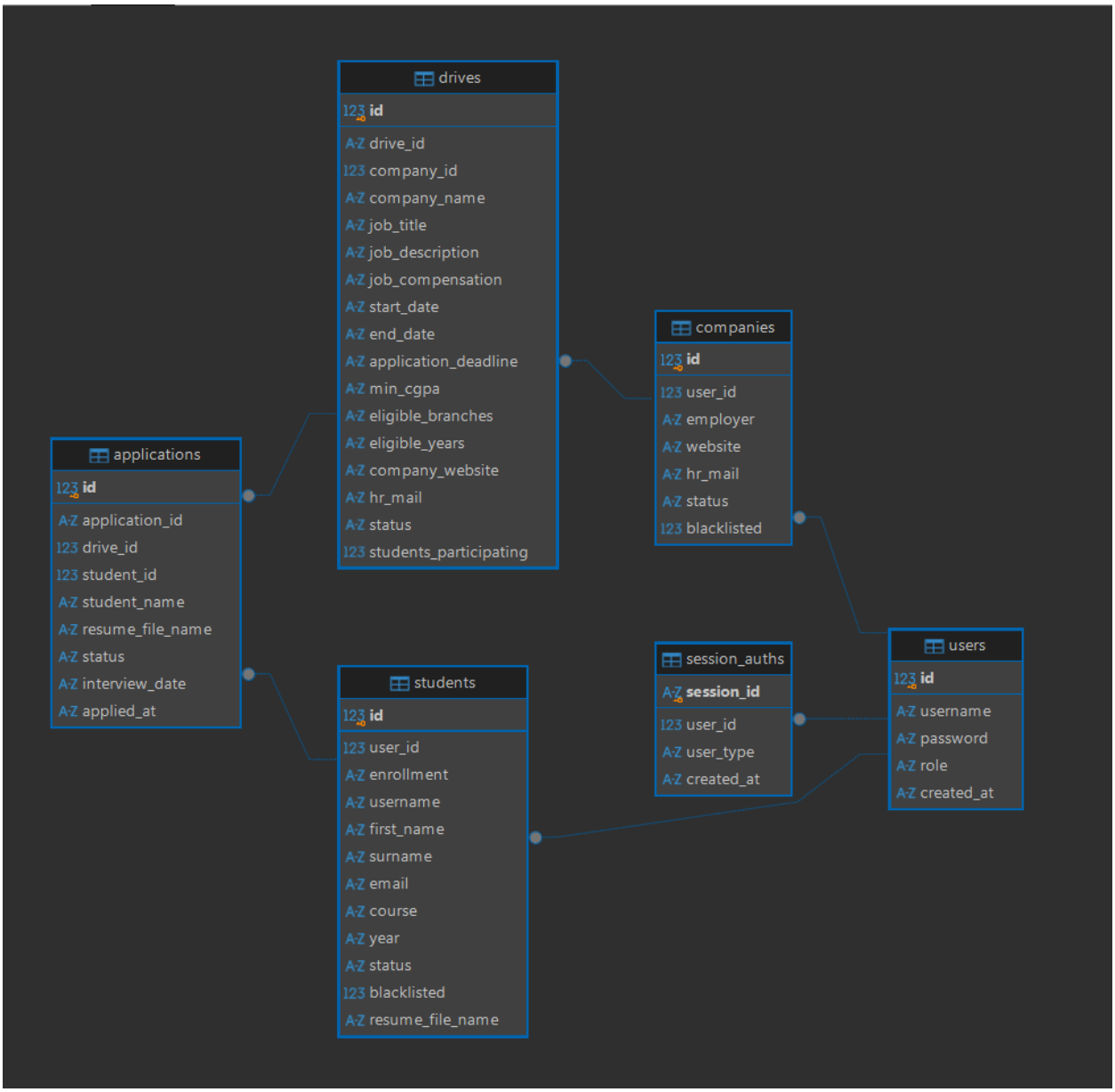
---

4. Technologies and Frameworks Used

Layer / Subsystem	Technology Used	Description & Purpose
Backend Framework	Flask (Python 3.12)	Core micro-framework for handling HTTP routing, REST API endpoints, and request middleware.
Database ORM	Flask-SQLAlchemy	Object-Relational Mapping library for defining models, entity relations, and executing database queries.
Session Cache	Redis (Port 6379)	In-memory key-value store used for sub-millisecond session authentication token lookup.
Background Queue	Celery	Asynchronous task queue runner used for generating CSV exports and compiling monthly HTML performance reports.
Frontend Framework	Vue.js 3 (Options/Composition)	Reactive component framework for building responsive user interface views and reactive state bindings.
Build Tool	Vite	Next-generation frontend build tool providing fast HMR (Hot Module Replacement) and optimized production bundles.
Database Storage	SQLite 3	Lightweight relational database engine storing application persistent state.
Styling & Design	Vanilla CSS3	Custom design system using CSS Grid, Flexbox, glassmorphism aesthetics, and clean micro-animations.

5. Database Schema / ER Diagram

The system database model (placement.db) consists of six core relational entities:



Tables & Fields:

- **User:** id (PK), username (Unique), password (Hashed bcrypt), role ('admin', 'company', 'student')
- **Company:** id (PK), user_id (FK -> users.id, Unique), employer, website, hr_mail, status ('requested', 'approved', 'active', 'inactive', 'denied'), blacklisted (Boolean)

- **Student:** id (PK), user_id (FK -> users.id, Unique), enrollment (Unique), username, first_name, surname, email, course, year, status ('Active', 'Inactive', 'Placed', 'denied'), blacklisted (Boolean), resume_file_name
- **Drive:** id (PK), drive_id (Unique 'DRV001'), company_id (FK -> companies.id), company_name, job_title, job_description, job_compensation, start_date, end_date, application_deadline, min_cgpa, status ('Pending', 'Approved', 'Rejected'), students_participating
- **Application:** id (PK), application_id (Unique 'APP301'), drive_id (FK -> drives.id), student_id (FK -> students.id), student_name, resume_file_name, interview_date, status ('Applied', 'Shortlisted', 'Selected', 'Rejected'), applied_at
- **SessionAuth:** id (PK), session_id (Unique), user_id (FK -> users.id), user_type, created_at

---

6. API Resource Endpoints

Authentication Endpoints

- **POST /api/auth/register:** Registers new company or student accounts with field validation.
- **POST /api/auth/login:** Authenticates credentials, creates a session token, caches it in Redis, and returns user profile metadata.
- **POST /api/auth/logout:** Invalidates and deletes the active session token from Redis and SQLite.

Admin Resource Endpoints

- **GET /api/admin/dashboard:** Fetches site-wide overview statistics, pending approvals, company registries, student registries, and placement report summaries.
- **PATCH /api/admin/companies/<name>:** Updates company approval status (approved/denied) or toggles blacklist status.
- **PATCH /api/admin/students/<enrollment>:** Updates student record details, profile status, or blacklist status.
- **PATCH /api/admin/drives/<drive_id>:** Approves or rejects placement drive submissions.

Company Endpoints

- **GET /api/company/dashboard:** Fetches company profile metrics, created drives, and received applicant records.
- **POST /api/company/drives:** Creates a new placement drive submission (restricted to active companies).
- **PATCH /api/company/profile:** Self-toggles company status between active and inactive.
- **PATCH /api/applications/<app_id>:** Updates application status (Shortlisted, Selected, Rejected) and sets interview date/time schedules.

- **POST /api/company/report/generate:** Dispatches a Celery task to generate a monthly performance HTML report.

Student Endpoints

- **GET /api/student/dashboard:** Retrieves approved placement drives, active applications, and student profile details.
- **PATCH /api/student/profile:** Updates student contact details, first name, surname, email, enrollment ID, and profile status (Active/Inactive).
- **POST /api/drives/<drive_code>/apply:** Submits a job application to an approved drive (restricted to active students with uploaded resumes).

Asynchronous Data Export Endpoints

- **POST /api/export/csv:** Dispatches a background Celery task to generate CSV exports for any active dashboard tab.
- **GET /api/export/status/<task_id>:** Returns Celery task execution status (PENDING, SUCCESS, FAILURE).
- **GET /api/export/download/<task_id>:** Serves the completed CSV file download.

---

7. Architecture and Features

Architecture Overview

The application consists of a main entry point file (main.py) which initializes the Flask application. The models directory contains database schema definitions. The api.py module handles different REST API resource endpoints and user requests. Vue 3 components in frontend/ src/ components are used for rendering frontend pages, and services/api.js provides centralized asynchronous HTTP communications.

Key Implemented Features

- **1. Multi-Role RBAC:** Custom middleware (auth_user) verifies request headers (X-Session-Id), validates active session keys in Redis, and enforces role permissions (admin, company, student).
- **2. Student Profile Management:** Allows students to update mandatory profile fields including First Name, Surname, Email, and Enrollment ID.
- **3. Interview Scheduling & Selection:** Companies can set candidate interview dates/times via native datetime controls and update applicant statuses through action controls (Shortlist, Select, Reject). Selecting a candidate automatically updates student status to Placed.
- **4. Active Status Guards:** Companies can self-toggle status (ACTIVE / INACTIVE). Drive creation is strictly restricted to active companies. Student drive application is restricted to active students.

- **5. Real-time Placement Reports:** Dynamically computes placement rates, approved/rejected drive ratios, and placed candidate totals.
- **6. Asynchronous Export Jobs:** Background CSV and report generation via Celery and Redis with instant fallbacks.

---

8. Video Presentation & Source Code

GitHub Repository: https://github.com/Gregnald/adv-placed

Video Walkthrough Drive Link: https://drive.google.com/drive/folders/1SxyngJ2mvApTh1lv-EvCXhSB5CAW-HiJ?usp=sharing